

EMBEDDED SYSTEM AND DESIGN

EXPERIMENT NO. 1

REAL-TIME BASKETBALL SCOREBOARD

Name: *Raphael S. Nano*

Course/Section: *CPE161P / AI*

Date of Performance: *November 14-18, 2023*

Date of Submission: *November 18, 2023*

Ma'am Cyrel Manlises
Instructor

DISCUSSION

Basketball is a sport played on a rectangular court by two teams of five players each, generally indoors. Each side attempts to score by tossing the ball through the opposing goal, which is an elevated horizontal hoop and net known as a basket. Shooting at different areas of the court would mean different points. If the shooter scores while outside the 3-point line, their team gets 3 points. If the shooter scores within the 3-point line, their team gets 2 points. Free throws, on the other hand, gives them 1 point. Knowing that, a real-time basketball scoreboard will be constructed and programmed, wherein 1, 2, or 3 points could be added or deducted every time a ball went in the basket.

The materials utilized to construct the real-time basketball scoreboard are the Arduino UNO, the 4-digit seven segment display, and 5 switch buttons.

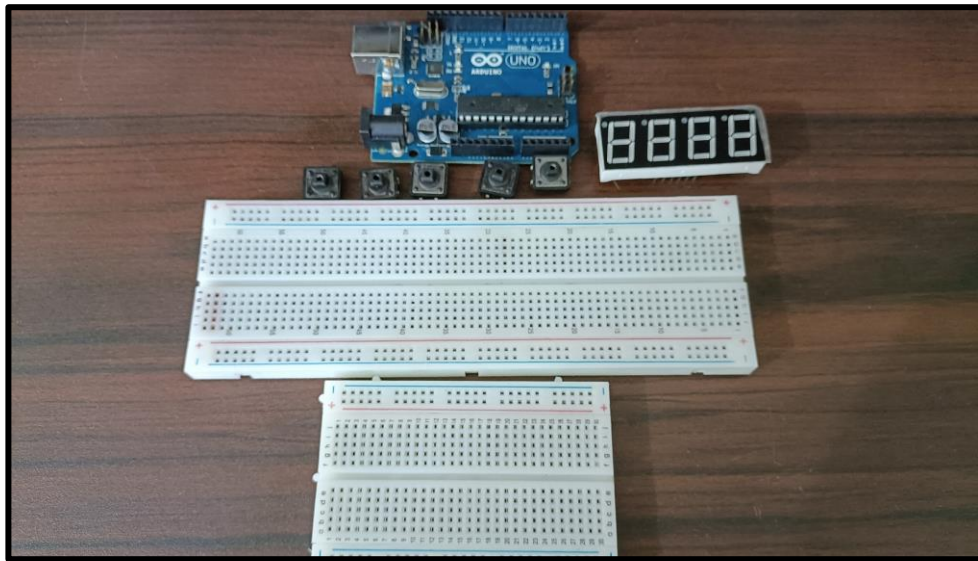


Figure 1. Materials for the Experiment

The first experiment primarily aims to demonstrate how a scoreboard for a basketball game works in real-time. Hence, there should be no delay in adjusting or changing the values of the seven-segment display every time a button was pushed down.

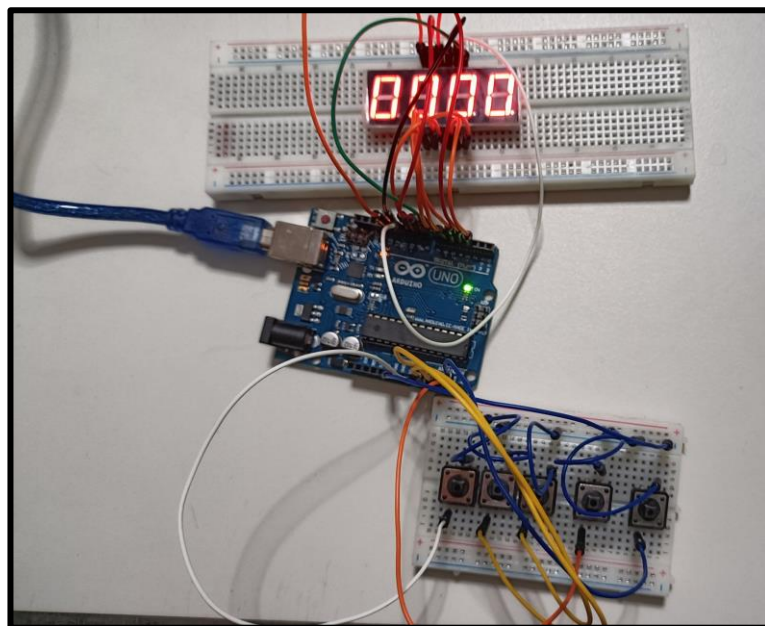


Figure 2. Final Output - Real-Time Basketball Scoreboard

Two functions were used for the adjusting of points since there are two teams to be handled. So, *team1()* function is used for the adding or subtracting of values for the first two digits of the 4-digit seven segment display. On the other hand, *team2()* function is used to adjust the values for the rightmost two digits for the seven segment display. Although there are two separate functions, their purpose is the same, and only the digits to be considered are different.

```

28 void team1() {
29     unsigned x = (k / 1000U) % 10;
30     unsigned y = (k / 100U) % 10;
31     if (digitalRead(A3) == 1) {
32         if (digitalRead(A0) == 0) {
33             if ((x == 9) && (y == 9)) {
34
35             } else {
36                 k = k + 100;
37
38                 sevseg.setNumber(k);
39             }
40             msdelay();
41             return;
42         } else if (digitalRead(A1) == 0) {
43             if ((x == 9) && (y == 9)) {
44
45             } else {
46                 k = k + 200;
47
48                 sevseg.setNumber(k);
49             }
50             msdelay();
51             return;
52         } else if (digitalRead(A2) == 0) {
53             if ((x == 9) && (y == 9)) {
54
55             } else {
56                 k = k + 300;
57
58                 sevseg.setNumber(k);
59             }
60             msdelay();
61             return;
62         }

```

```

99 void team2() {
100     unsigned x = (k / 10U) % 10;
101     unsigned y = (k / 1U) % 10;
102     if (digitalRead(A3) == 1) {
103         if (digitalRead(A0) == 0) {
104             if ((x != 9) || (y != 9)) {
105                 k = k + 1;
106                 sevseg.setNumber(k);
107             }
108             msdelay();
109             return;
110         } else if (digitalRead(A1) == 0) {
111             if ((x == 9) && (y == 9)) {
112
113             } else {
114                 k = k + 2;
115
116                 sevseg.setNumber(k);
117             }
118             msdelay();
119             return;
120         } else if (digitalRead(A2) == 0) {
121             if ((x == 9) && (y == 9)) {
122
123             } else {
124                 k = k + 3;
125
126                 sevseg.setNumber(k);
127             }
128             msdelay();
129             return;
130         }

```

Figure 3. Codes for *team1()* and *team2()* functions

In setting up the proper scoring of the game, the handler must be able to adjust the points of

the teams by 1s, 2s, or 3s, depending on the shot made by the player. This is done using the `adjust()` function. The concept behind the point adjustment begins with the first three buttons, which represents 1 point, 2 points, and 3 points, respectively. The user should press the 5th button in order to point the other team. Hence, the 5th button serves as a switch from one team to another. Since errors in scoring are possible, there is also the 4th button, which handles the decrementing of the points. So, in order to decrement the points for team 1, the 4th button should be held first before pressing either one of the first 3 buttons. To decrement the points for team 2, the same procedure should be done. All of these principles utilize a `digitalRead` and input pullups so that the button will be triggered when pressed.

```

168 void adjust(int teamNum) {
169     if (teamNum == 1) {
170         team1();
171     }
172     else if (teamNum == 2) {
173         team2();
174     }
175 }

```

Figure 4. Code for adjust(teamNum) - combining team1() and team2()

The seven-segment display utilizes an external library named SevSeg.h. The number of digits, the digit pins, the segment pins, and other hardware configuration was first programmed in the initialization stage in order to activate the 4-digit seven segment display. Moreover, only the functions setNumber() and refreshDisplay() were used so that the component would work appropriately.

```

byte numDigits = 4;
byte digitPins[] = { 13, 12, 11, 10 };
byte segmentPins[] = { 9, 8, 7, 6, 5, 4, 3, 2 };
bool resistorOnSegments = false;
byte hardwareConfig = COMMON_CATHODE;
sevseg.begin(hardwareConfig, numDigits, digitPins, segmentPins,
resistorOnSegments, 0, 1, 0);
sevseg.setBrightness(90);
sevseg.setNumber(0);

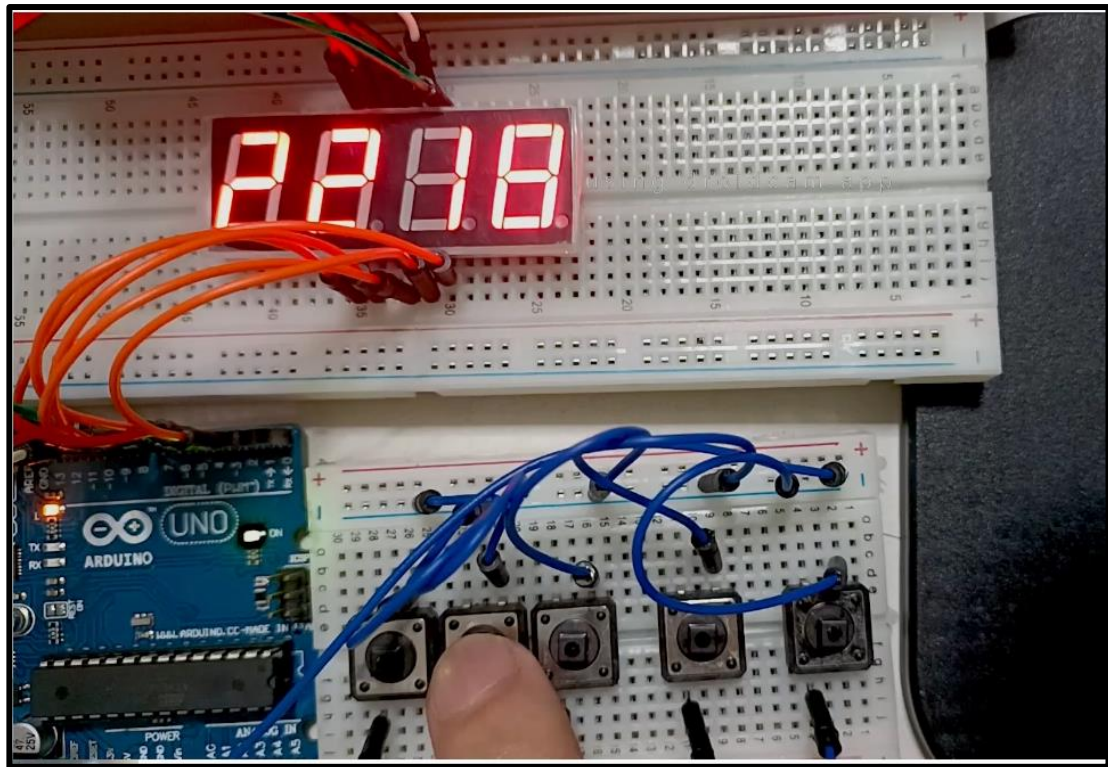
```

Figure 5. Code for pushDown() functions

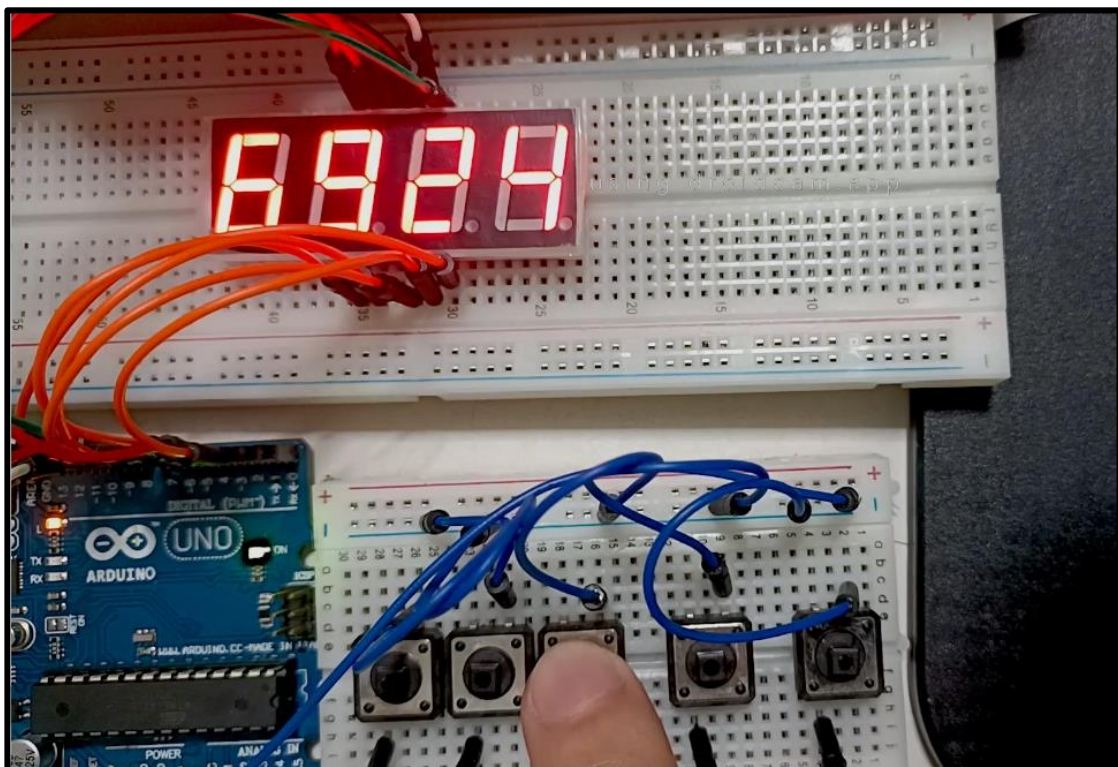
Lastly, the loop of the program itself begins by refreshing the seven-segment display. Afterwards, the system checks if the 5th button, which is the team-switch button, is pressed so that the system would know what points for a team is being adjusted. Lastly, the `adjust(teamNum)` function is called so that in case that the user has pressed an additional or decremental button, the value of the seven-segment display would be adjusted.

```
188 void loop() {  
189     sevseg.refreshDisplay();  
190     if (digitalRead(A4) == 0) {  
191         if (teamNum == 1) {  
192             teamNum++;  
193             msdelay();  
194         } else if (teamNum == 2) {  
195             teamNum--;  
196             msdelay();  
197         }  
198     }  
199  
200     adjust(teamNum);  
201 }
```

Figure 6. Experiment 1 Program Loop



(a)



(b)

Figure 7. Sample Out put (a) and (b)

CONCLUSION

To conclude, for the first objective, the real-time basketball scoreboard basically works by scoring 1, 2, or 3 points by using three buttons. A 4th button would be held first in order to decrement the points using the first three buttons as well. To score points for the second team, the 5th button should be pressed once to switch teams. Through series connections, pin layouts, and sensor and motor triggering, the usage of microprocessors, logic circuits, and switching theory is illustrated. The real-time scoreboard for a basketball game was successfully developed and presented in a basic handshaking as a consequence of employing many variables,

modules, and calling such functions to construct an embedded system.